

Trabalho 2

Árvores binárias

Relatório parcial - seções 1, 2 e 3

Técnicas de Programação Avançada
Professor: Victorio Albani de Carvalho

Integrantes

Bernardo Simão Rosa

Matheus Abreu

Levi Monteiro

Conteúdo desta entrega

1. Desenvolvimento
2. Análise matemática de complexidade
3. Análise empírica de complexidade

1. Desenvolvimento

1.1 Biblioteca e aplicativo

A biblioteca de árvores binárias de busca foi implementada em Java. A classe `ArvoreBinaria<T>` herda de `ArvoreBinariaBase<T>`, fornecida pelo professor. Generics permite escolher o tipo dos objetos, e o `Comparator` recebido no construtor define a chave de comparação. O aplicativo usa duas árvores de contatos, uma indexada por nome e outra por telefone.

A inserção segue a regra ensinada nas aulas: valores menores vão para a esquerda; maiores ou iguais vão para a direita. A busca segue um único caminho. A remoção trata nós sem filhos, com um filho e com dois filhos. Neste último caso, usa o menor valor da subárvore direita, uma das opções dos slides. Não há rotações: a ordem da entrada determina o formato da árvore.

Foram implementados adicionar, pesquisar, remover, `quantidadeNos`, altura e os caminhamentos em ordem e em nível. O `toString` é herdado da classe base. A altura conta arestas: -1 na árvore vazia e 0 quando há apenas a raiz. Os percursos usam laços, pilha ou fila explícita para permitir cadeias profundas sem depender de recursão.

O aplicativo do trabalho de listas foi reaproveitado. Na abertura, permite escolher lista não ordenada (1), lista ordenada (2) ou árvore binária (3). Os índices continuam declarados como `IColecao<Contato>`. A remoção por referência no índice de nomes preserva o contato correto quando existem nomes iguais. Um `HashSet` valida telefones únicos e libera o número após a exclusão.

Na lista encadeada, um campo `total` passa a guardar a quantidade de elementos. Cada inserção bem-sucedida incrementa o contador, e cada remoção bem-sucedida o decrementa, inclusive por referência. Assim, `quantidadeNos` passa de $\Theta(n)$ para $\Theta(1)$, sem mudar a ordem dos elementos. A árvore mantém a contagem por percurso, com custo $\Theta(n)$.

1.2 Participação e repositório

Integrante	Responsabilidade no grupo
Bernardo Simão Rosa	Biblioteca de árvores binárias
Matheus Abreu	Adaptação do aplicativo de contatos
Levi Monteiro	Relatório e experimentos

Repositório do projeto: <https://github.com/bs20242/tpa-trabalho2-arvores>. O README descreve a organização dos pacotes e a compilação com JDK 21. O programa de teste da biblioteca, os geradores e os benchmarks ficam no pacote testes.

1.3 Uso de inteligência artificial

A Inteligência Artificial auxiliou na implementação e na revisão do código, na preparação dos testes e na elaboração das análises. Também ajudou na organização do relatório, das tabelas e dos gráficos. Os tempos apresentados foram medidos em execuções locais do benchmark.

2. Análise matemática de complexidade

2.1 Critérios da análise

As linhas numeradas a seguir correspondem ao arquivo `ArvoreBinaria.java` deste projeto. No Modelo Simplificado de Custos, atribuímos custo constante a comparação, acesso a referência e atribuição. Seja n o número de nós, h a altura em arestas e d o número de nós visitados em uma procura. Para strings de comprimento variável L , a comparação pode custar $O(L)$; as análises abaixo consideram chaves de tamanho limitado.

Na árvore perfeita, $n = 2^{(h+1)} - 1$, logo $h = \log_2(n+1) - 1$. Na árvore completamente degenerada, $h = n - 1$. Esses formatos podem aparecer na mesma biblioteca, porque ela não faz balanceamento automático.

2.2 adicionar

```

19     public boolean adicionar(T valor) {
20         if (valor == null) return false;
21         No<T> novo = new No<>(valor);
22         if (raiz == null) { raiz = novo; return true; }
23         No<T> atual = raiz;
24         while (true) {
25             if (comparador.compare(valor, atual.valor) < 0) {
26                 if (atual.esq == null) { atual.esq = novo; return true; }
27                 atual = atual.esq;
28             } else {
29                 if (atual.dir == null) { atual.dir = novo; return true; }
30                 atual = atual.dir;
31             }
32         }
33     }

```

Linhas	Contagem e raciocínio
19, 28, 31-33	Assinatura e delimitadores de blocos; não percorrem nós.
20	Uma validação de valor nulo: $\Theta(1)$.
21	Uma criação de nó: $\Theta(1)$.
22	Um teste da raiz. Se estiver vazia, liga o nó e retorna.
23	Uma atribuição para iniciar a procura na raiz.
24-25	d iterações e d comparações para escolher um lado.
26 e 29	Um teste de filho por iteração; na última, insere e retorna.
27 e 30	$d - 1$ avanços no total, descendo um nível de cada vez.

$T(n, h) = a + b \cdot d$, com $d \leq h + 1$ para árvore não vazia. O pior caso ocorre ao inserir abaixo de uma folha mais profunda. O custo é $\Theta(h + 1)$: $\Theta(\log n)$ na perfeita e $\Theta(n)$ na degenerada. Inserir na árvore vazia custa $\Theta(1)$.

2. Análise matemática | pesquisa

2.3 pesquisar

```

36     public T pesquisar(T valor) {
37         if (valor == null) return null;
38         No<T> atual = raiz;
39         while (atual != null) {
40             int c = comparador.compare(valor, atual.valor);
41             if (c == 0) return atual.valor;
42             atual = c < 0 ? atual.esq : atual.dir;
43         }
44         return null;
45     }

```

Linhas	Contagem e raciocínio
36, 43 e 45	Assinatura e delimitadores.
37	Uma validação de valor nulo.
38	Uma atribuição: a procura começa na raiz.
39	Até $d + 1$ testes, incluindo o teste final de nó nulo.
40	d comparações com a chave de cada nó visitado.
41	d testes de igualdade; retorna se encontrar.
42	Até d avanços, escolhendo apenas um dos filhos.
44	Um retorno de null somente quando a busca não encontra a chave.

$T(n, h) = a + b \cdot d$. Buscar uma folha mais profunda, ou uma chave ausente que percorra um caminho de altura máxima, produz $\Theta(h + 1)$. Assim, o pior caso é $\Theta(\log n)$ na perfeita e $\Theta(n)$ na degenerada. Encontrar a raiz custa $\Theta(1)$.

Relação entre altura e trabalho da busca

Formato	Altura	Nós visitados na folha mais profunda
Perfeita	$\log_2(n + 1) - 1$	$\log_2(n + 1)$
Degenerada	$n - 1$	n

Com 16.383 nós, o pior caminho da árvore perfeita visita 14 nós. Na degenerada, visita 16.383. Na perfeita, cada escolha descarta uma subárvore grande. Na degenerada, quase todos os nós ficam no mesmo caminho, como em uma lista encadeada.

2. Análise matemática | remoção

2.4 remover e seu auxiliar

```

48     public boolean remover(T valor) {
49         return removerInterno(valor, false);
50     }

54     private boolean removerInterno(T valor, boolean referencia) {
55         if (valor == null) return false;
56         No<T> pai = null, atual = raiz;
57         while (atual != null) {
58             int c = comparador.compare(valor, atual.valor);
59             if (c == 0 && (!referencia || atual.valor == valor)) break;
60             pai = atual;
61             atual = c < 0 ? atual.esq : atual.dir;
62         }
63         if (atual == null) return false;
64         if (atual.esq != null && atual.dir != null) {
65             No<T> paiSucessor = atual, sucessor = atual.dir;
66             while (sucessor.esq != null) {
67                 paiSucessor = sucessor;
68                 sucessor = sucessor.esq;
69             }
70             atual.valor = sucessor.valor;
71             if (paiSucessor == atual) paiSucessor.dir = sucessor.dir;
72             else paiSucessor.esq = sucessor.dir;
73         } else {
74             No<T> filho = atual.esq != null ? atual.esq : atual.dir;
75             if (pai == null) raiz = filho;
76             else if (pai.esq == atual) pai.esq = filho;
77             else pai.dir = filho;
78         }
79         return true;
80     }

```

Linhas	Contagem e raciocínio
48-50 e 54	remover chama o auxiliar uma vez; assinatura e delimitadores.
55-56	Uma validação e uma inicialização das referências.
57-61	d visitas: compara, verifica a chave e avança um nível.
62-64	Fecha o laço, testa se encontrou e escolhe o caso de remoção.
65	Inicia a procura do sucessor, apenas no caso de dois filhos.
66-68	s avanços no caminho esquerdo da subárvore direita.
69-72	Fecha o laço, copia o valor e religa o filho do sucessor: $\Theta(1)$.
73-77	No caso de zero ou um filho, religa o pai ou a raiz: $\Theta(1)$.
78-80	Fecha os blocos e retorna true.

$T(n, h) = a + b \cdot d + c \cdot s$. A procura e a descida até o sucessor compõem um caminho limitado pela altura. O pior caso é $\Theta(h + 1)$: $\Theta(\log n)$ na perfeita e $\Theta(n)$ na degenerada. Remover uma folha mais profunda já atinge esse custo. A remoção por referência usa o mesmo auxiliar, distinguindo a instância do objeto quando há nomes iguais.

2. Análise matemática | contagem

2.5 quantidadeNos da árvore

```

83     public int quantidadeNos() {
84         if (raiz == null) return 0;
85         Deque<No<T>> pilha = new ArrayDeque<>();
86         pilha.push(raiz);
87         int total = 0;
88         while (!pilha.isEmpty()) {
89             No<T> atual = pilha.pop();
90             total++;
91             if (atual.dir != null) pilha.push(atual.dir);
92             if (atual.esq != null) pilha.push(atual.esq);
93         }
94         return total;
95     }

```

Linhas	Contagem e raciocínio
83, 93 e 95	Assinatura e delimitadores.
84	Um teste de raiz vazia; retorna 0 nesse caso.
85-87	Uma criação de pilha, uma inclusão da raiz e uma inicialização.
88	$n + 1$ testes de término.
89-90	n retiradas da pilha e n incrementos do contador local.
91-92	n testes de cada filho e $n - 1$ inclusões na pilha ao todo.
94	Um retorno da contagem obtida.

$T(n) = a + b \cdot n + c \cdot (n - 1)$, portanto $\Theta(n)$ na perfeita e na degenerada. Cada nó entra e sai da pilha uma vez. ArrayDeque tem custo constante amortizado para essas operações. A contagem da árvore não usa o contador permanente acrescentado à lista.

2.6 Síntese dos piores casos

Método da árvore	Perfeita	Degenerada
adicionar	$\Theta(\log n)$	$\Theta(n)$
pesquisar	$\Theta(\log n)$	$\Theta(n)$
remover	$\Theta(\log n)$	$\Theta(n)$
quantidadeNos	$\Theta(n)$	$\Theta(n)$

Na carga crescente, a inserção do i -ésimo nó compara com $i - 1$ nós. A soma de 0 até $n - 1$ é $n(n - 1)/2$: montagem $\Theta(n^2)$. Inserir pelas medianas produz altura logarítmica, com montagem $\Theta(n \log n)$. A leitura e a validação de telefones com HashSet acrescentam custo esperado $\Theta(n)$, considerando registros de tamanho limitado. Montar dois índices mantém essas ordens.

Na lista não ordenada, inserir no início custa $\Theta(1)$, e a carga com HashSet tem custo esperado $\Theta(n)$. Na lista ordenada, a carga pode custar $\Theta(n^2)$. Consultar a quantidade na lista agora custa $\Theta(1)$; isso não torna a busca ou a remoção constantes.

3. Análise empírica de complexidade

3.1 Implementação dos experimentos

GeradorDadosArvores produz arquivos UTF-8 no formato nome;telefone. Foram usados quatro tamanhos: 2.047, 4.095, 8.191 e 16.383 contatos. Cada tamanho tem duas versões, totalizando oito arquivos. Os tamanhos são iguais a $2^k - 1$ para permitir árvores perfeitas, com todas as folhas no mesmo nível, conforme a definição dos slides. A escolha mantém a carga degenerada repetida em tempo viável; o enunciado recomenda, mas não exige, os mesmos tamanhos do trabalho de listas.

Cada registro usa um identificador único: nome Contato XXXXXX e telefone com 11 dígitos. A ordem crescente por nome coincide com a ordem por telefone. Na entrada degenerada, os identificadores vão de 1 até n . Na entrada perfeita, são escritos primeiro a mediana do intervalo e depois, recursivamente, as medianas de seus intervalos esquerdo e direito. A árvore é formada pelas inserções normais, sem balanceamento automático.

Após a carga, folhaMaisDistante faz um percurso em nível e seleciona o último nó visitado. Ele é uma folha no nível mais profundo e define o alvo da busca por telefone. Em seguida, pesquisa-se o nome do mesmo contato e remove-se seu telefone. As alturas, profundidades e quantidades foram verificadas antes das medidas. Nesta entrada controlada, os dois índices têm o mesmo formato, e o alvo também é o pior caso por nome; essa coincidência não é garantida com nomes reais em outra ordem.

n	Altura perfeita	Altura degenerada	Visitas: perfeita / degenerada
2.047	10	2.046	11 / 2.047
4.095	11	4.094	12 / 4.095
8.191	12	8.190	13 / 8.191
16.383	13	16.382	14 / 16.383

Ambiente e intervalos de medição

Coleta realizada em 05/10/2026, em Windows, processador Intel(R) Core(TM) i5-10400F CPU @ 2.90GHz, 31,9 GB de RAM e Java Corretto 21.0.6. A JVM usou heap inicial de 256 MB e máximo de 1.024 MB. Houve aquecimento com dez cargas de 2.047 contatos e buscas, fora dos intervalos medidos.

Foram feitas três execuções por tamanho e formato, com cadastro novo a cada carga: 24 execuções. A carga mede abertura, leitura, criação dos contatos, validação e montagem dos dois índices. As buscas e a remoção medem uma chamada ao respectivo método com System.nanoTime(). Escolher o alvo, calcular a altura, preparar as chaves e sincronizar a remoção por nome ficam fora desses intervalos. As tabelas mostram as medianas, em milissegundos.

Após cada exclusão, os dois índices foram verificados com $n - 1$ nós e o telefone removido deixou de ser encontrado. Os testes da biblioteca passaram em 25.139 verificações; o contador da lista passou em outras 20.000 verificações aleatórias. Os testes anteriores e os roteiros do menu nos três modos também passaram.

3. Análise empírica | resultados

3.2 Tempos coletados e interpretação

Tabela 1. Medianas da leitura e montagem dos dois índices.

Contatos	Perfeita (ms)	Degenerada (ms)
2.047	16,334	56,083
4.095	3,529	215,955
8.191	6,271	829,293
16.383	11,605	3.290,817

Tabela 2. Medianas das operações no mesmo contato, na folha mais profunda.

Formato	Contatos	Telefone (ms)	Nome (ms)	Remoção (ms)
Perfeita	2.047	0,0010	0,0006	0,0015
Perfeita	4.095	0,0017	0,0008	0,0015
Perfeita	8.191	0,0009	0,0005	0,0016
Perfeita	16.383	0,0011	0,0005	0,0016
Degenerada	2.047	0,0456	0,0334	0,0768
Degenerada	4.095	0,0656	0,0646	0,1499
Degenerada	8.191	0,1162	0,1174	0,3252
Degenerada	16.383	0,3245	0,3329	0,2920

Leitura dos resultados

Ao quase dobrar o número de contatos, a mediana da carga degenerada aumentou 3,85x, 3,84x, 3,97x. O crescimento fica próximo do previsto para a soma quadrática das comparações. A árvore perfeita teve carga menor nos quatro tamanhos. Seus caminhos curtos evitam a soma quadrática, mas a carga ainda inclui leitura, alocação, validação e dois índices.

As buscas e a remoção na perfeita visitam de 11 a 14 nós nos tamanhos usados; na degenerada, de 2.047 a 16.383. Os tempos curtos oscilam por compilação da JVM, cache, coleta de lixo e agendamento do sistema. Há também valores de carga que não crescem de forma monotônica. Três repetições e quatro tamanhos não bastam para ajustar uma lei de crescimento com precisão. A contagem dos caminhos e a análise matemática explicam as ordens; os tempos mostram o comportamento desta execução.

Os CSVs preservam cada medição, sem remover valores extremos. A mediana de três medidas é o valor central após ordenar o trio. Escolher a folha e contar os nós não entra no tempo das buscas nem no da remoção.

3. Análise empírica | gráficos

Gráficos das medianas

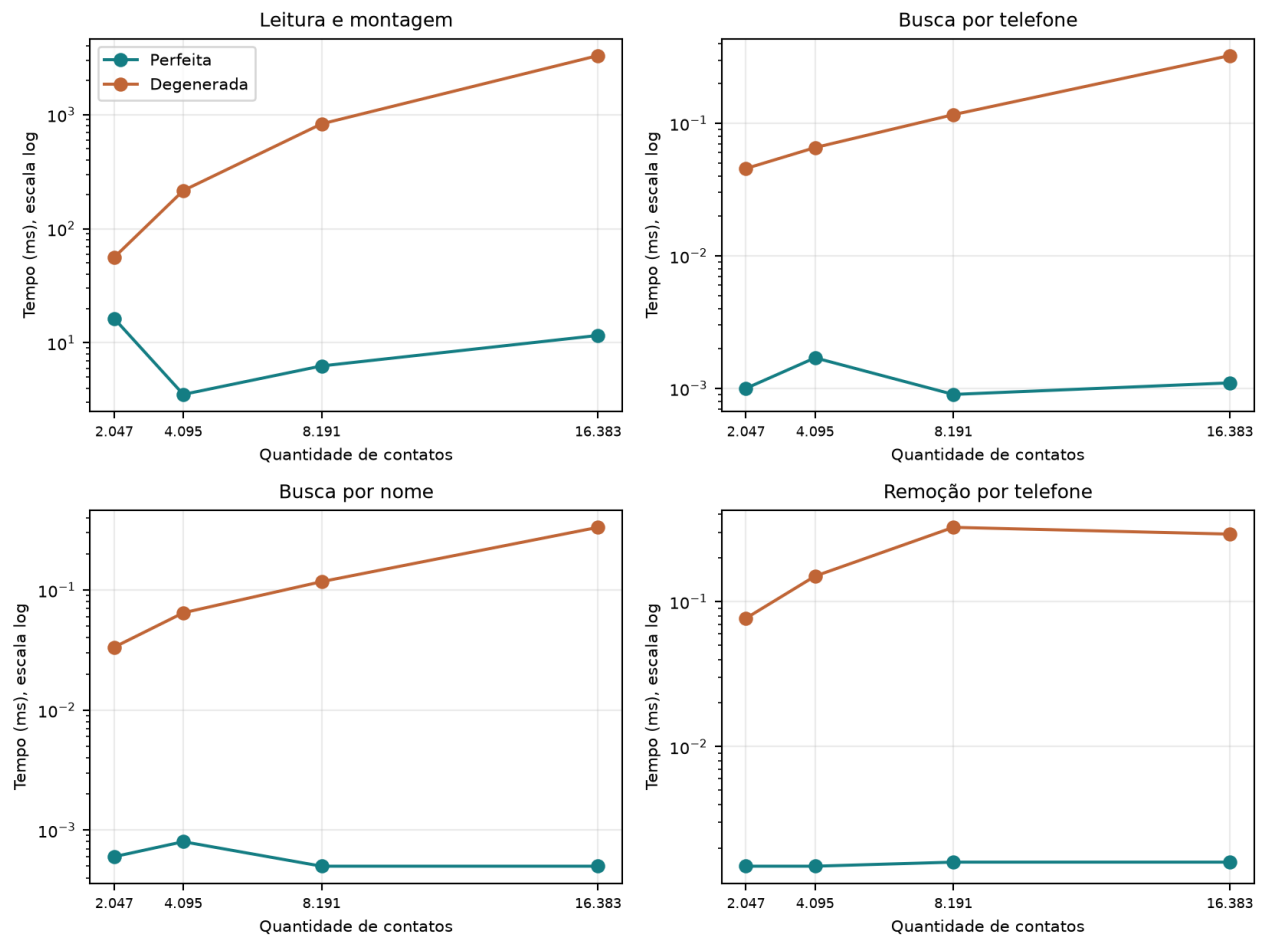


Figura 1. Os pontos foram calculados com as mesmas medianas das tabelas 1 e 2. A escala vertical logarítmica permite mostrar os dois formatos. As linhas apenas ligam pontos medidos; não representam outras execuções.

Efeito do formato da árvore

A entrada crescente gera uma cadeia mesmo usando a biblioteca correta. Nesse formato, uma busca ou remoção no fim do caminho exige $\Theta(n)$, como o percurso de uma lista. A entrada por medianas mantém a altura pequena e permite $\Theta(\log n)$ para essas operações. Isso mostra que a eficiência depende da ordem de entrada enquanto não há balanceamento automático.

A contagem de elementos da lista agora é constante, mas não altera a topologia da árvore nem o número de comparações na busca. Também não muda os tempos antigos: a contagem estava fora dos intervalos cronometrados do benchmark de listas.

3. Análise empírica | comparação com listas

Resultados do trabalho anterior

Tabela 3. Tempos em milissegundos registrados no trabalho 1. São da coleta anterior, com outros tamanhos e entradas.

Lista	Contatos	Carga	Telefone	Nome	Remoção
Não ordenada	10.000	34,49	1,9462	1,3518	1,0386
Não ordenada	25.000	59,18	4,1928	3,8690	3,9615
Não ordenada	50.000	89,27	6,9765	2,1509	7,6943
Não ordenada	100.000	145,98	6,0489	2,0732	8,4620
Ordenada	10.000	961,56	0,5913	0,5995	0,7271
Ordenada	25.000	8.590,54	2,8232	3,3659	3,9314
Ordenada	50.000	65.166,86	5,6915	5,1778	9,1309
Ordenada	100.000	252.445,47	10,6781	9,3485	16,6710

A lista não ordenada insere no início em $\Theta(1)$ e, com HashSet para validar telefones, tem carga esperada $\Theta(n)$. A lista ordenada procura a posição de cada contato e pode atingir $\Theta(n^2)$ na carga. Nas duas listas, buscar ou remover o elemento do fim custa $\Theta(n)$. A árvore degenerada tem a mesma ordem de percurso; a perfeita reduz o pior caminho a $\Theta(\log n)$.

Nesta coleta, a carga da árvore perfeita com 16.383 contatos teve mediana de 11,605 ms. No trabalho anterior, a lista ordenada com 10.000 contatos levou 961,56 ms. Os resultados mostram diferenças entre as coletas, mas não permitem afirmar um fator de velocidade controlado: tamanhos, entradas e aquecimento são diferentes. A comparação de ordens é sustentada pela análise dos algoritmos.

Materiais e arquivos usados

Foram usados o enunciado Trabalho 2 - Árvores Binárias.pdf, itens 1 a 5; os slides APNP - Semana 3 - Arvores.pdf, especialmente altura, busca, caminhamentos e os três casos de remoção; e o exemplo ArvBin.zip. O exemplo C usa recursão e o maior da subárvore esquerda; esta versão Java usa percursos iterativos e o menor da direita, outra opção dos slides. A árvore vazia retorna -1 conforme o contrato Java fornecido pelo professor.

A classe base e a interface são do [repositório do professor](#). O endereço tpa_2026_1 do enunciado redireciona para tpa_bsi. O CSV histórico foi obtido do [trabalho 1 entregue](#). O CSV atual está em dados/resultados_arvores.csv; ambiente e verificação das entradas estão em ambiente.json e manifesto.json.